CGC UNIVERSITY, MOHALI

Department of Computer Science & Engineering

PPF WEEK 9 (MINI PROJECT)

Campus Route Finder Using Graphs

Data Structures and Algorithms (DSA) - Project Report

Submitted by	Dhruv
Roll Number	2556447
Programme	B.Tech CSE
PPF Activity	Data Structures Mini Project - Campus Route Finder Using Graphs
Assessment Method	Mini Project (20 Marks)
Mode	Individual
Submission	GitHub Repository
Duration	1 Week
Language / IDE	C (GCC) / Visual Studio Code

1. Problem Statement

The aim of this project is to build a **Campus Route Finder** in C in which campus locations are represented as **vertices** and the paths between them as **edges**. The application lets a user explore connected locations, find a route between two places, check reachability and compare route-finding approaches. The project demonstrates graph representation (adjacency list built from linked lists, and adjacency matrix), Breadth-First Search (BFS), Depth-First Search (DFS) and shortest-path logic for an unweighted graph, together with a time and space complexity analysis.

2. Requirement Coverage (Tasks vs Implementation)

Task	Requirement	How it is implemented
1	Graph of at least 8 locations	10 locations: Main Gate, Parking, Library, Block 1, Block 2, Cafeteria, Auditorium, Lab, Hostel, Sports Complex (9 edges + 1 edge for the separate part).
2	Adjacency list (+ optional matrix)	`head[]` / `tail[]` arrays of linked lists (`AdjNode`) are the main structure; `build_matrix()` creates a V x V adjacency matrix. Menu options 1 and 2.
3	BFS minimum-edge route	`bfs_route()` uses an array-based queue, a visited array and a parent array.
4	DFS reachability	`dfs_reachable()` (recursive) and `dfs_is_reachable()` (iterative, explicit stack).
5	User enters source and destination	Menu option 3 reads two names (case-insensitive) and prints the BFS route and number of edges.
6	Reachable and disconnected locations	Menu options 4 and 6; `disconnected_from()` and `print_components()`.
7	Complexity explanation	Section 6 of this report.
8	At least 3 test cases	Section 7 (4 test cases; also available at menu option 7).

3. Data Structure Design and Graph Representation

The campus is modelled as an **undirected, unweighted graph** G = (V, E). Every path can be walked in both directions, so each edge is stored twice in the adjacency list. Location names are mapped to integer indices (0 to V-1), and a `CampusGraph` struct holds the names, the linked-list heads and tails, and the matrix. Nodes are created with `malloc()` and released by `free_graph()`. Hostel and Sports Complex are deliberately kept as a separate component so that the *unreachable destination* case can be demonstrated.

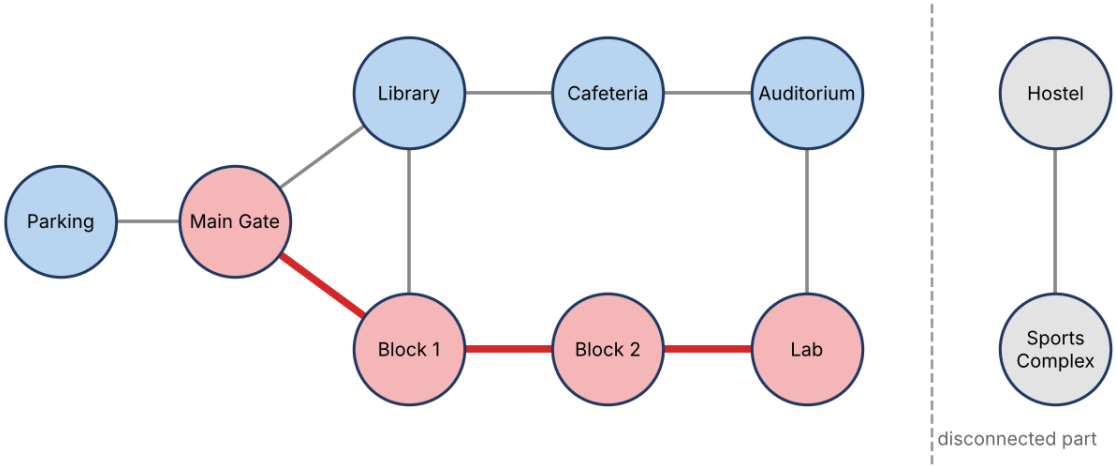


Figure 1: Campus graph. The red edges show the BFS route Main Gate - Block 1 - Block 2 - Lab. Grey nodes form a disconnected component.

3.1 Adjacency list (program output)

TERMINAL - bash

```
ADJACENCY LIST
------------------------------------------------------------
Main Gate      -> Parking, Library, Block 1
Parking        -> Main Gate
Library        -> Main Gate, Block 1, Cafeteria
Block 1        -> Main Gate, Library, Block 2
Cafeteria      -> Library, Auditorium
Block 2        -> Block 1, Lab
Lab            -> Block 2, Auditorium
Auditorium     -> Cafeteria, Lab
Hostel         -> Sports Complex
Sports Complex -> Hostel
```

3.2 Adjacency matrix (program output)

TERMINAL - bash

```
ADJACENCY MATRIX  (1 = path exists, 0 = no path)
------------------------------------------------------------
  [0] Main Gate
  [1] Parking
  [2] Library
  [3] Block 1
  [4] Cafeteria
  [5] Block 2
  [6] Lab
  [7] Auditorium
  [8] Hostel
  [9] Sports Complex

     0   1   2   3   4   5   6   7   8   9
[0]  0   1   1   1   0   0   0   0   0   0
[1]  1   0   0   0   0   0   0   0   0   0
[2]  1   0   0   1   1   0   0   0   0   0
[3]  1   0   1   0   0   1   0   0   0   0
[4]  0   0   1   0   0   0   0   1   0   0
[5]  0   0   0   1   0   0   1   0   0   0
[6]  0   0   0   0   0   1   0   1   0   0
[7]  0   0   0   0   1   0   1   0   0   0
[8]  0   0   0   0   0   0   0   0   0   1
[9]  0   0   0   0   0   0   0   0   1   0
```

The matrix is symmetric because the graph is undirected. In the matrix, the entry [i][j] = 1 means that a direct path exists between location i and location j.

4. Algorithms

4.1 BFS - shortest route (minimum number of edges)

BFS explores the graph level by level using a queue. In an unweighted graph the first time BFS reaches a vertex is through a path with the fewest edges, so the route found is a shortest path. A parent (predecessor) array records from which vertex each location was discovered; the route is rebuilt by walking back from the destination to the source and reversing the list.

pseudocode

```
BFS(source, dest):
  visited[source] = 1; parent[source] = -1; queue = [source]
  while queue is not empty:
    node = queue.dequeue()
    if node == dest: break
    for each neighbour of node:
      if neighbour not visited:
        visited[neighbour] = 1; parent[neighbour] = node; queue.enqueue(neighbour)
  if dest never discovered: return NO ROUTE
  path = []; cur = dest
  while cur != -1: path.append(cur); cur = parent[cur]
```

```pseudocode (continued)
  return reverse(path)
```

4.2 DFS - reachability and traversal

DFS goes as deep as possible along one branch before backtracking. It uses the recursion stack (or an explicit stack in the iterative version) and a visited array. Running DFS from a starting location gives every reachable location; all remaining locations are disconnected from it. Repeating DFS from each unvisited vertex gives the connected components.

```pseudocode
DFS(node):
  visited[node] = 1; output(node)
  for each neighbour of node:
    if neighbour not visited: DFS(neighbour)
```

4.3 Difference between BFS and DFS

Point	BFS	DFS
Data structure	Queue (FIFO)	Stack / recursion (LIFO)
Shortest path (unweighted)	Yes - guaranteed minimum edges	No - may return a longer path
Order of visiting	Level by level	Deep along a branch, then backtrack
Best used for	Shortest route, nearest location	Reachability, connected components, cycle detection
Time / Space	O(V + E) / O(V)	O(V + E) / O(V)

5. Sample Input and Output

5.1 BFS route: Main Gate to Lab (menu option 3)

```
Input: Source = Main Gate, Destination = Lab
BFS Route : Main Gate -> Block 1 -> Block 2 -> Lab
Minimum edges : 3
```

5.2 DFS traversal from Main Gate (menu option 4)

```
DFS reachable : Main Gate -> Parking -> Library -> Block 1 -> Block 2 -> Lab -> Auditorium -> Cafeteria
```

5.3 Reachability using DFS (menu option 5)

```
Input: Source = Main Gate, Destination = Hostel
Hostel is NOT reachable from Main Gate
```

5.4 Disconnected locations (menu option 6)

```
Reachable : Main Gate, Parking, Library, Block 1, Block 2, Lab, Auditorium, Cafeteria
Disconnected : Hostel, Sports Complex
Components :
  1) Main Gate, Parking, Library, Block 1, Block 2, Lab, Auditorium, Cafeteria
  2) Hostel, Sports Complex
  Total connected components: 2
```

Observation: DFS visited Parking first because it is the first neighbour of Main Gate, which shows that DFS order depends on adjacency-list order, while BFS gave the 3-edge route Main Gate - Block 1 - Block 2 - Lab.

6. Time and Space Complexity

Let V = number of locations (vertices) and E = number of paths (edges).

Algorithm / Operation	Adjacency List	Adjacency Matrix
Space to store the graph	O(V + E)	$O(V^2)$

Algorithm / Operation	Adjacency List	Adjacency Matrix
BFS time	O(V + E) - each vertex is enqueued once and each edge examined twice	$O(V^2)$ - scanning a full row of V entries for every vertex
DFS time	O(V + E)	$O(V^2)$
BFS / DFS extra space	O(V) - visited, parent, queue / stack arrays	O(V)
Check if edge (u, v) exists	O(deg(u))	O(1)
List all neighbours of a vertex	O(deg(u))	O(V)
Add an edge	O(1) (tail pointer append; O(deg) if duplicate check is done)	O(1)
Best for	Sparse graphs (campus map: few paths per place)	Dense graphs, frequent edge-existence queries

Why the adjacency list is chosen: a campus map is sparse (each location connects to only 2-3 others, so E is much smaller than V^2). The adjacency list therefore uses less memory and makes BFS and DFS run in O(V + E). The matrix is implemented only for comparison. Path reconstruction costs O(V) in the worst case, which does not change the overall O(V + E) of BFS.

7. Test Cases (Expected vs Actual)

ID	Test case	Input (Src -> Dest)	Expected output	Actual output	Result
TC1	Direct route	Main Gate -> Library	Main Gate -> Library	Main Gate -> Library	PASS
TC2	Multi-stop route	Main Gate -> Lab	Main Gate -> Block 1 -> Block 2 -> Lab	Main Gate -> Block 1 -> Block 2 -> Lab	PASS
TC3	Unreachable destination	Main Gate -> Hostel	No route (NULL)	No route (NULL)	PASS
TC4	Same source and destination	Library -> Library	Library	Library	PASS

Raw program output of menu option 7:

```
TERMINAL - bash

$ ./campus_route_finder

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
1. Show all locations (Adjacency List)
2. Show Adjacency Matrix
3. Find route between two places (BFS)
4. Show reachable locations from a start (DFS)
5. Check if destination is reachable (DFS)
6. Show disconnected locations / components
7. Run test cases
0. Exit
Enter choice: 7

============================================================
TEST CASES
============================================================

TC1 Direct route: Main Gate -> Library
  Expected : Main Gate -> Library
  Actual   : Main Gate -> Library
  Result   : PASS

TC2 Multi-stop route: Main Gate -> Lab
  Expected : Main Gate -> Block 1 -> Block 2 -> Lab
  Actual   : Main Gate -> Block 1 -> Block 2 -> Lab
  Result   : PASS

TC3 Unreachable destination: Main Gate -> Hostel
```


TERMINAL - bash (continued)
Expected : No route (NULL)
Actual : No route (NULL)
Result : PASS
TC4 Same source & destination: Library -> Library
Expected : Library
Actual : Library
Result : PASS
CAMPUS ROUTE FINDER (Graph - BFS / DFS)
[menu options 1-7 and 0 as shown above]
Enter choice: 0
Thank you!

8. Source Code and Program Output (VS Code)

The code is written in campus_route_finder.c, compiled with GCC (`gcc -Wall -Wextra -o campus_route_finder campus_route_finder.c`, no warnings) and executed from the VS Code terminal. Each block below shows the source code or the actual output of the menu option mentioned.

8.1 Source code - graph struct (adjacency list and adjacency matrix)

File: campus_route_finder.c (lines 11-116)

campus_route_finder.c - Visual Studio Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_V    32
#define NAME_LEN 32

/* Adjacency list node: one entry per neighbour */
typedef struct AdjNode {
    int v;
    struct AdjNode *next;
} AdjNode;

typedef struct {
    int      n;                     /* number of vertices */
    char     names[MAX_V][NAME_LEN];  /* index -> location name */
    AdjNode *head[MAX_V];           /* adjacency list heads */
    AdjNode *tail[MAX_V];           /* tails keep insertion order */
    int      matrix[MAX_V][MAX_V];    /* adjacency matrix */
} CampusGraph;

/* ---------- Graph construction ---------- */
static int find_location(const CampusGraph *g, const char *name) {
    for (int i = 0; i < g->n; i++)
        if (strcmp(g->names[i], name) == 0) return i;
    return -1;
}

static int add_location(CampusGraph *g, const char *name) {
    int i = find_location(g, name);
    if (i >= 0) return i;
    if (g->n >= MAX_V) { fprintf(stderr, "Too many locations\n"); exit(1); }
    strncpy(g->names[g->n], name, NAME_LEN - 1);
    g->names[g->n][NAME_LEN - 1] = '\0';
    g->head[g->n] = g->tail[g->n] = NULL;
    return g->n++;
}

static int has_edge_list(const CampusGraph *g, int a, int b) {
    for (AdjNode *p = g->head[a]; p; p = p->next)
        if (p->v == b) return 1;
    return 0;
}

static void append_neighbour(CampusGraph *g, int a, int b) {
    AdjNode *node = (AdjNode *)malloc(sizeof(AdjNode));
    if (!node) { fprintf(stderr, "Out of memory\n"); exit(1); }
    node->v = b; node->next = NULL;
    if (g->tail[a]) g->tail[a]->next = node; else g->head[a] = node;
    g->tail[a] = node;
}

/* Add an undirected edge between two locations. */
static void add_path(CampusGraph *g, const char *a, const char *b) {
    int u = add_location(g, a), v = add_location(g, b);
    if (!has_edge_list(g, u, v)) {
        append_neighbour(g, u, v);
        append_neighbour(g, v, u);
    }
}

/* Build adjacency matrix from the adjacency list. */
static void build_matrix(CampusGraph *g) {
    memset(g->matrix, 0, sizeof(g->matrix));
```

campus_route_finder.c - Visual Studio Code (continued)

```c
    for (int a = 0; a < g->n; a++)
        for (AdjNode *p = g->head[a]; p; p = p->next)
            g->matrix[a][p->v] = 1;
}

static void free_graph(CampusGraph *g) {
    for (int i = 0; i < g->n; i++) {
        AdjNode *p = g->head[i];
        while (p) { AdjNode *t = p->next; free(p); p = t; }
        g->head[i] = g->tail[i] = NULL;
    }
}

/* ---------- Display ---------- */
static void show_adj_list(const CampusGraph *g) {
    printf("\nADJACENCY LIST\n");
    for (int i = 0; i < 60; i++) putchar('-');
    putchar('\n');
    for (int i = 0; i < g->n; i++) {
        printf("%-14s -> ", g->names[i]);
        for (AdjNode *p = g->head[i]; p; p = p->next)
            printf("%s%s", g->names[p->v], p->next ? ", " : "");
        putchar('\n');
    }
}

static void show_adj_matrix(CampusGraph *g) {
    build_matrix(g);
    printf("\nADJACENCY MATRIX  (1 = path exists, 0 = no path)\n");
    for (int i = 0; i < 60; i++) putchar('-');
    putchar('\n');
    for (int i = 0; i < g->n; i++) printf("  [%d] %s\n", i, g->names[i]);
    printf("\n    ");
    for (int i = 0; i < g->n; i++) printf(" %-3d", i);
    putchar('\n');
    for (int i = 0; i < g->n; i++) {
        printf("[%d]  ", i);
        for (int j = 0; j < g->n; j++) printf("%-3d%s", g->matrix[i][j], j + 1 < g->n ? " " : "");
        putchar('\n');
    }
}
```

File: campus_route_finder.c - build_campus() and show_route() (lines 218-244)

campus_route_finder.c - Visual Studio Code

```c
/* ---------- Campus data ---------- */
static void build_campus(CampusGraph *g) {
    static const char *edges[][2] = {
        {"Main Gate", "Parking"}, {"Main Gate", "Library"},
        {"Main Gate", "Block 1"}, {"Library", "Block 1"},
        {"Library", "Cafeteria"}, {"Block 1", "Block 2"},
        {"Block 2", "Lab"},       {"Cafeteria", "Auditorium"},
        {"Auditorium", "Lab"},
        /* separate (disconnected) part of the campus */
        {"Hostel", "Sports Complex"},
    };
    memset(g, 0, sizeof(*g));
    for (size_t i = 0; i < sizeof(edges) / sizeof(edges[0]); i++)
        add_path(g, edges[i][0], edges[i][1]);
}

static void show_route(const CampusGraph *g, int s, int d) {
    int path[MAX_V], plen;
    int steps = bfs_route(g, s, d, path, &plen);
    if (steps < 0) {
        printf("No route found from %s to %s (unreachable / invalid).\n", g->names[s], g->names[d]);
    } else {
        printf("BFS Route : ");
        print_names(g, path, plen, " -> ");
        printf("\nMinimum edges : %d\n", steps);
    }
}
```

8.2 Source code - BFS and DFS

File: campus_route_finder.c (lines 118-180)

campus_route_finder.c - Visual Studio Code

```c
/* ---------- BFS : shortest route (min edges) ----------
 * Fills path[] (source ... dest) and returns the number of edges,
 * or -1 if either location is invalid or dest is unreachable. */
static int bfs_route(const CampusGraph *g, int src, int dest, int *path, int *plen) {
    if (src < 0 || dest < 0 || src >= g->n || dest >= g->n) { *plen = 0; return -1; }
    int visited[MAX_V] = {0}, parent[MAX_V], queue[MAX_V];
    int qh = 0, qt = 0;
    for (int i = 0; i < g->n; i++) parent[i] = -2;   /* -2 = never discovered */
    visited[src] = 1; parent[src] = -1;              /* -1 = no parent (source) */
    queue[qt++] = src;
    while (qh < qt) {
        int node = queue[qh++];
        if (node == dest) break;
        for (AdjNode *p = g->head[node]; p; p = p->next) {
            if (!visited[p->v]) {
                visited[p->v] = 1;
                parent[p->v] = node;
                queue[qt++] = p->v;
            }
        }
    }
    if (parent[dest] == -2) { *plen = 0; return -1; }
    /* path reconstruction using parent / predecessor */
    int tmp[MAX_V], k = 0;
    for (int cur = dest; cur != -1; cur = parent[cur]) tmp[k++] = cur;
    for (int i = 0; i < k; i++) path[i] = tmp[k - 1 - i];   /* reverse */
    *plen = k;
    return k - 1;
}

/* ---------- DFS ---------- */
static void dfs_visit(const CampusGraph *g, int node, int *visited, int *order, int *cnt) {
    visited[node] = 1;
    order[(*cnt)++] = node;
    for (AdjNode *p = g->head[node]; p; p = p->next)
        if (!visited[p->v]) dfs_visit(g, p->v, visited, order, cnt);
}

/* Recursive DFS: fills order[] with all locations reachable from start. */
static int dfs_reachable(const CampusGraph *g, int start, int *order) {
    int visited[MAX_V] = {0}, cnt = 0;
    if (start >= 0 && start < g->n) dfs_visit(g, start, visited, order, &cnt);
    return cnt;
}

/* Iterative DFS (explicit stack): is dest reachable from src? */
static int dfs_is_reachable(const CampusGraph *g, int src, int dest) {
    if (src < 0 || dest < 0 || src >= g->n || dest >= g->n) return 0;
    int visited[MAX_V] = {0}, stack[MAX_V * MAX_V], top = 0;
    stack[top++] = src;
    while (top > 0) {
        int node = stack[--top];
        if (node == dest) return 1;
        if (!visited[node]) {
            visited[node] = 1;
            int nb[MAX_V], c = 0;
            for (AdjNode *p = g->head[node]; p; p = p->next) nb[c++] = p->v;
            for (int i = c - 1; i >= 0; i--)          /* reversed: keeps list order */
                if (!visited[nb[i]]) stack[top++] = nb[i];
        }
    }
    return 0;
}
```

File: campus_route_finder.c (lines 182-216)

campus_route_finder.c - Visual Studio Code

```c
static void print_names(const CampusGraph *g, const int *idx, int n, const char *sep) {
    for (int i = 0; i < n; i++) printf("%s%s", g->names[idx[i]], i + 1 < n ? sep : "");
}

/* Locations that cannot be reached from start. */
static int disconnected_from(const CampusGraph *g, int start, int *out) {
    int order[MAX_V], reach[MAX_V] = {0}, c = 0;
    int cnt = dfs_reachable(g, start, order);
    for (int i = 0; i < cnt; i++) reach[order[i]] = 1;
    for (int i = 0; i < g->n; i++) if (!reach[i]) out[c++] = i;
    return c;
}

static void print_components(const CampusGraph *g) {
    int seen[MAX_V] = {0}, order[MAX_V], k = 0;
    for (int i = 0; i < g->n; i++) {
        if (seen[i]) continue;
        int cnt = dfs_reachable(g, i, order);
        for (int j = 0; j < cnt; j++) seen[order[j]] = 1;
        printf("  %d) ", ++k);
        print_names(g, order, cnt, ", ");
        putchar('\n');
    }
}

static void print_comp_count(const CampusGraph *g) {
    int seen[MAX_V] = {0}, order[MAX_V], k = 0;
    for (int i = 0; i < g->n; i++) {
        if (seen[i]) continue;
        int cnt = dfs_reachable(g, i, order);
        for (int j = 0; j < cnt; j++) seen[order[j]] = 1;
        k++;
    }
    printf("  Total connected components: %d\n", k);
}
```

8.3 Output - compile, graph input: adjacency list and matrix (options 1 and 2)

```
TERMINAL - bash

$ gcc -Wall -Wextra -o campus_route_finder campus_route_finder.c
$ ./campus_route_finder

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
1. Show all locations (Adjacency List)
2. Show Adjacency Matrix
3. Find route between two places (BFS)
4. Show reachable locations from a start (DFS)
5. Check if destination is reachable (DFS)
6. Show disconnected locations / components
7. Run test cases
0. Exit
Enter choice: 1

ADJACENCY LIST
------------------------------------------------------------
Main Gate      -> Parking, Library, Block 1
Parking        -> Main Gate
Library        -> Main Gate, Block 1, Cafeteria
Block 1        -> Main Gate, Library, Block 2
Cafeteria      -> Library, Auditorium
Block 2        -> Block 1, Lab
Lab            -> Block 2, Auditorium
Auditorium     -> Cafeteria, Lab
Hostel         -> Sports Complex
Sports Complex -> Hostel

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
   [menu options 1-7 and 0 as shown above]
Enter choice: 2

ADJACENCY MATRIX  (1 = path exists, 0 = no path)
------------------------------------------------------------
  [0] Main Gate
  [1] Parking
  [2] Library
  [3] Block 1
  [4] Cafeteria
  [5] Block 2
  [6] Lab
  [7] Auditorium
  [8] Hostel
  [9] Sports Complex

     0   1   2   3   4   5   6   7   8   9
[0]  0   1   1   1   0   0   0   0   0   0
[1]  1   0   0   0   0   0   0   0   0   0
[2]  1   0   0   1   1   0   0   0   0   0
[3]  1   0   1   0   0   1   0   0   0   0
[4]  0   0   1   0   0   0   0   1   0   0
[5]  0   0   0   1   0   0   1   0   0   0
[6]  0   0   0   0   0   1   0   1   0   0
[7]  0   0   0   0   1   0   1   0   0   0
[8]  0   0   0   0   0   0   0   0   0   1
[9]  0   0   0   0   0   0   0   0   1   0

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
   [menu options 1-7 and 0 as shown above]
Enter choice: 0
Thank you!
```

8.4 Output - BFS route with user input (option 3)

```
TERMINAL - bash

$ ./campus_route_finder

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
1. Show all locations (Adjacency List)
2. Show Adjacency Matrix
3. Find route between two places (BFS)
4. Show reachable locations from a start (DFS)
5. Check if destination is reachable (DFS)
6. Show disconnected locations / components
7. Run test cases
0. Exit
Enter choice: 3
Locations: Main Gate, Parking, Library, Block 1, Cafeteria, Block 2, Lab, Auditorium, Hostel, Sports Complex
Source: Main Gate
Destination: Lab
BFS Route : Main Gate -> Block 1 -> Block 2 -> Lab
Minimum edges : 3

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
   [menu options 1-7 and 0 as shown above]
Enter choice: 3
Locations: Main Gate, Parking, Library, Block 1, Cafeteria, Block 2, Lab, Auditorium, Hostel, Sports Complex
Source: Main Gate
Destination: Hostel
No route found from Main Gate to Hostel (unreachable / invalid).

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
   [menu options 1-7 and 0 as shown above]
Enter choice: 0
Thank you!
```

8.5 Output - DFS traversal and reachability (options 4 and 5)

```
TERMINAL - bash

$ ./campus_route_finder

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
1. Show all locations (Adjacency List)
2. Show Adjacency Matrix
3. Find route between two places (BFS)
4. Show reachable locations from a start (DFS)
5. Check if destination is reachable (DFS)
6. Show disconnected locations / components
7. Run test cases
0. Exit
Enter choice: 4
Start location: Main Gate
DFS reachable : Main Gate -> Parking -> Library -> Block 1 -> Block 2 -> Lab -> Auditorium -> Cafeteria

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
   [menu options 1-7 and 0 as shown above]
Enter choice: 5
Source: Main Gate
Destination: Hostel
Hostel is NOT reachable from Main Gate

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
   [menu options 1-7 and 0 as shown above]
Enter choice: 0
Thank you!
```

8.6 Output - disconnected locations (option 6)

```
TERMINAL - bash

$ ./campus_route_finder

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
1. Show all locations (Adjacency List)
2. Show Adjacency Matrix
3. Find route between two places (BFS)
4. Show reachable locations from a start (DFS)
5. Check if destination is reachable (DFS)
6. Show disconnected locations / components
7. Run test cases
0. Exit
Enter choice: 6
Start location: Main Gate
Reachable    : Main Gate, Parking, Library, Block 1, Block 2, Lab, Auditorium, Cafeteria
Disconnected : Hostel, Sports Complex
Components   :
  1) Main Gate, Parking, Library, Block 1, Block 2, Lab, Auditorium, Cafeteria
  2) Hostel, Sports Complex
  Total connected components: 2

============================================================
        CAMPUS ROUTE FINDER (Graph - BFS / DFS)
============================================================
   [menu options 1-7 and 0 as shown above]
Enter choice: 0
Thank you!
```

8.7 Output - test cases (option 7)

All 4 test cases PASS (full output shown in Section 7).

Note: in the terminal transcripts, the menu is printed in full once; later repetitions of the identical menu are abbreviated to save space.

9. Conclusion

The Campus Route Finder successfully represents a campus as a graph in C using an adjacency list (linked lists) and an adjacency matrix. BFS with parent tracking finds the route with the minimum number of edges, DFS answers reachability queries and exposes disconnected locations, and all four test cases passed. Both traversals run in O(V + E) time and O(V) extra space with an adjacency list. Future scope: adding distances as weights and using Dijkstra's algorithm, a GUI / map view, and handling one-way or blocked paths.

GitHub Repository Link: _____ (paste your link here)